

CS3807 – Deep Learning Laboratory

Experiment 1

Implementation of a Single Layer Perceptron for Binary Classification

1 Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. The experiment also studies the learning behaviour of the perceptron, evaluates its classification performance, compares different learning rates, compares Step and Sigmoid activation functions, investigates the XOR limitation, compares the custom model with the Scikit-learn Perceptron, and studies the effect of feature normalization on convergence.

2 Background Theory

2.1 Artificial Neuron

An artificial neuron is the fundamental computational unit of a neural network. It receives multiple input features, multiplies each input by a corresponding weight, adds a bias term, and applies an activation function to produce an output.

For an input vector

$$\mathbf{x} = [x_1, x_2, \dots, x_n]^T$$

and weight vector

$$\mathbf{w} = [w_1, w_2, \dots, w_n]^T,$$

the weighted sum is calculated as

$$z = \mathbf{w}^T \mathbf{x} + b$$

where b represents the bias term.

2.2 Single Layer Perceptron

A Single Layer Perceptron is a supervised learning algorithm used for binary classification. It produces a binary output by applying an activation function to the weighted sum of the input features.

The predicted output is

$$\hat{y} = f(z)$$

where $f(z)$ is the activation function.

2.3 Perceptron Learning Rule

If a training sample is misclassified, the weights are updated using

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

and the bias is updated using

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where

- η is the learning rate,
- y is the actual class label,
- $\hat{y}$ is the predicted class label.

2.4 Linear Separability

A Single Layer Perceptron can correctly converge only when the classes are linearly separable. This means that a single straight line in two dimensions, or a single hyperplane in higher dimensions, must be capable of separating the classes.

3 Activation Functions

3.1 Step Activation Function

The Step activation function is defined as

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step function produces a hard binary output and is therefore suitable for the classical Single Layer Perceptron used in this experiment.

3.2 Sigmoid Activation Function

The Sigmoid activation function is defined as

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Unlike the Step function, the Sigmoid function produces continuous values between 0 and 1.

3.3 Step vs Sigmoid Comparison

The Step activation function gives a direct binary output of 0 or 1, whereas the Sigmoid function provides a smooth continuous output between 0 and 1. The Step function is discontinuous at $z = 0$, has zero derivative almost everywhere, and does not provide useful gradients for backpropagation.

The Sigmoid function is differentiable and can therefore support gradient-based optimization. However, Sigmoid can suffer from vanishing gradients for very large positive or negative inputs. Modern deep neural networks therefore commonly use other activation functions such as ReLU in hidden layers.

4 Dataset Description

The Banknote Authentication Dataset is used for this experiment. The dataset contains numerical features extracted from images of genuine and forged banknotes.

Property	Description
Dataset	Banknote Authentication Dataset
Number of Instances	1372
Number of Features	4
Number of Classes	2
Missing Values	None
Task	Binary Classification

Table 1: Dataset Summary

4.1 Feature Description

The four numerical input features are

- Variance
- Skewness
- Curtosis
- Entropy

4.2 Target Class

The target variable contains two classes:

- Class 0 – Authentic Banknote
- Class 1 – Forged Banknote

5 Experimental Procedure

5.1 Dataset Exploration

The dataset was loaded and inspected by displaying the first five samples. Dataset dimensions, missing values, descriptive statistics, data types, and class distribution were examined.

5.2 Exploratory Data Analysis

Exploratory Data Analysis was performed using

- Feature histograms
- Correlation heatmap
- Scatter plot

- Boxplots

These visualizations were used to study feature distributions, relationships among variables, class distribution, and potential outliers.

5.3 Data Preprocessing

The input features and target labels were separated. The dataset was split into 80% training data and 20% testing data.

The numerical input features were standardized using

$$x_{scaled} = \frac{x - \mu}{\sigma}$$

where μ is the training-feature mean and σ is the training-feature standard deviation.

5.4 Perceptron Implementation

The Single Layer Perceptron was implemented from scratch with

- Weight initialization
- Bias initialization
- Step activation function
- Forward propagation
- Prediction
- Error calculation
- Perceptron weight update rule
- Bias update rule

5.5 Model Training

The model was trained for a fixed number of epochs. During each epoch, every training sample was processed. For misclassified samples, the weights and bias were updated.

The following values were recorded:

- Epoch number
- Number of misclassified samples
- Updated weights
- Updated bias

5.6 Model Evaluation

The trained model was evaluated on unseen test data using

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

5.7 Learning Rate Study

The complete experiment was repeated using the learning rates

$$0.001, \quad 0.01, \quad 0.1$$

Separate fresh perceptron models were trained for each learning rate. Their convergence behaviour and classification performance were compared.

5.8 XOR Experiment

The XOR dataset was constructed using the four binary input combinations

$$(0, 0), \quad (0, 1), \quad (1, 0), \quad (1, 1)$$

with corresponding targets

$$0, \quad 1, \quad 1, \quad 0.$$

The Single Layer Perceptron was trained on the XOR dataset to demonstrate its inability to perfectly solve a non-linearly separable problem.

5.9 Normalization Study

Two fresh perceptron models were trained using identical learning rates, epochs, and train-test splits.

- Model 1 was trained without feature normalization.
- Model 2 was trained with feature normalization.

Their training-error curves and evaluation metrics were compared to study the effect of normalization on model convergence.

5.10 Scikit-learn Comparison

The custom Perceptron implemented from scratch was compared with the Perceptron implementation available in Scikit-learn. Both models were evaluated using Accuracy, Precision, Recall, and F1-score.

6 Source Code

6.1 Library Import and Dataset Loading

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5
6 from sklearn.model_selection import train_test_split
7 from sklearn.preprocessing import StandardScaler
8 from sklearn.metrics import (
9     accuracy_score,
10    precision_score,
11    recall_score,
12    f1_score,
13    confusion_matrix
14 )
15
16 column_names = [
17     "Variance",
18     "Skewness",
19     "Curtosis",
20     "Entropy",
21     "Class"
22 ]
23
24 df = pd.read_csv(
25     "data_banknote_authentication.txt",
26     header=None,
27     names=column_names
28 )
```

6.2 Data Preprocessing

```
1 feature_columns = [
2     "Variance",
3     "Skewness",
4     "Curtosis",
5     "Entropy"
6 ]
7
8 X = df[feature_columns].values
9 y = df["Class"].values
10
11 X_train, X_test, y_train, y_test = train_test_split(
12     X,
13     y,
14     test_size=0.20,
15     random_state=42,
16     stratify=y
17 )
```

```

18
19 scaler = StandardScaler()
20
21 X_train_scaled = scaler.fit_transform(X_train)
22 X_test_scaled = scaler.transform(X_test)

```

6.3 Single Layer Perceptron from Scratch

```

1 class SingleLayerPerceptron:
2
3     def __init__(self, learning_rate=0.01, epochs=50):
4         self.learning_rate = learning_rate
5         self.epochs = epochs
6         self.weights = None
7         self.bias = 0.0
8         self.errors_history = []
9         self.weights_history = []
10        self.bias_history = []
11
12    def activation(self, z):
13        return 1 if z >= 0 else 0
14
15    def predict_one(self, x):
16        z = np.dot(x, self.weights) + self.bias
17        return self.activation(z)
18
19    def fit(self, X, y):
20
21        self.weights = np.zeros(X.shape[1])
22        self.bias = 0.0
23
24        self.errors_history = []
25        self.weights_history = []
26        self.bias_history = []
27
28        for epoch in range(self.epochs):
29
30            misclassified = 0
31
32            for xi, target in zip(X, y):
33
34                prediction = self.predict_one(xi)
35                error = target - prediction
36
37                if error != 0:
38
39                    misclassified += 1
40
41                    self.weights += (
42                        self.learning_rate
43                        * error
44                        * xi
45                    )

```

```

46         self.bias += (
47             self.learning_rate
48             * error
49         )
50
51     self.errors_history.append(misclassified)
52     self.weights_history.append(
53         self.weights.copy()
54     )
55     self.bias_history.append(self.bias)
56
57     return self
58
59 def predict(self, X):
60
61     predictions = []
62
63     for x in X:
64         predictions.append(
65             self.predict_one(x)
66         )
67
68     return np.array(predictions)
69

```

6.4 Model Training

```

1 model = SingleLayerPerceptron(
2     learning_rate=0.01,
3     epochs=50
4 )
5
6 model.fit(
7     X_train_scaled,
8     y_train
9 )

```

6.5 Model Evaluation

```

1 y_pred = model.predict(X_test_scaled)
2
3 accuracy = accuracy_score(y_test, y_pred)
4
5 precision = precision_score(
6     y_test,
7     y_pred,
8     zero_division=0
9 )
10
11 recall = recall_score(
12     y_test,

```



```

13     y_pred,
14     zero_division=0
15 )
16
17 f1 = f1_score(
18     y_test,
19     y_pred,
20     zero_division=0
21 )
22
23 cm = confusion_matrix(
24     y_test,
25     y_pred
26 )

```

6.6 Learning Rate Comparison

```

1 learning_rates = [
2     0.001,
3     0.01,
4     0.1
5 ]
6
7 lr_models = {}
8
9 for lr in learning_rates:
10
11     temp_model = SingleLayerPerceptron(
12         learning_rate=lr,
13         epochs=50
14     )
15
16     temp_model.fit(
17         X_train_scaled,
18         y_train
19     )
20
21     lr_models[lr] = temp_model

```

6.7 Step and Sigmoid Comparison

```

1 def step_function(z):
2     return np.where(z >= 0, 1, 0)
3
4 def sigmoid_function(z):
5     return 1 / (1 + np.exp(-z))
6
7 z_values = np.linspace(
8     -10,
9     10,
10    500

```

```
11 )
12
13 step_outputs = step_function(z_values)
14
15 sigmoid_outputs = sigmoid_function(z_values)
```

6.8 XOR Experiment

```
1 X_xor = np.array([
2     [0, 0],
3     [0, 1],
4     [1, 0],
5     [1, 1]
6 ])
7
8 y_xor = np.array([
9     0,
10    1,
11    1,
12    0
13 ])
14
15 xor_model = SingleLayerPerceptron(
16     learning_rate=0.1,
17     epochs=20
18 )
19
20 xor_model.fit(
21     X_xor,
22     y_xor
23 )
24
25 xor_predictions = xor_model.predict(
26     X_xor
27 )
```

6.9 Normalization Study

```
1 model_without_normalization = (
2     SingleLayerPerceptron(
3         learning_rate=0.01,
4         epochs=50
5     )
6 )
7
8 model_without_normalization.fit(
9     X_train,
10    y_train
11 )
12
13 model_with_normalization = (
```

```

14     SingleLayerPerceptron(
15         learning_rate=0.01,
16         epochs=50
17     )
18 )
19
20 model_with_normalization.fit(
21     X_train_scaled,
22     y_train
23 )

```

6.10 Scikit-learn Comparison

```

1 from sklearn.linear_model import Perceptron
2
3 sk_model = Perceptron(
4     eta0=0.01,
5     max_iter=50,
6     random_state=42
7 )
8
9 sk_model.fit(
10     X_train_scaled,
11     y_train
12 )
13
14 sk_pred = sk_model.predict(
15     X_test_scaled
16 )

```

7 Experimental Results

7.1 Dataset Exploration Results

The dataset contained 1372 samples, four numerical input features, and one binary target variable. No missing values were present in the dataset.

The dataset was divided into training and testing subsets using an 80:20 ratio.

7.2 Main Perceptron Results

The final experimental values obtained after model training are shown below.

- Final Weights: *Insert actual values*
- Final Bias: *Insert actual value*
- Accuracy: *Insert actual value*
- Precision: *Insert actual value*
- Recall: *Insert actual value*
- F1-score: *Insert actual value*

7.3 Learning Rate Study Results

The learning rates 0.001, 0.01, and 0.1 were compared using their training-error behaviour and final test performance.

Insert actual experimental observations here.

7.4 XOR Experiment Results

The Single Layer Perceptron was unable to perfectly classify all XOR samples because the XOR classes are not linearly separable.

Insert actual XOR predictions and accuracy here.

7.5 Normalization Study Results

The models trained with and without normalization were compared using training errors and classification metrics.

Insert actual comparison results here.

7.6 Scikit-learn Comparison Results

The custom implementation and Scikit-learn Perceptron were compared using the same evaluation metrics.

Insert actual comparison results here.

8 Generated Plots with Interpretation

8.1 Feature Histograms

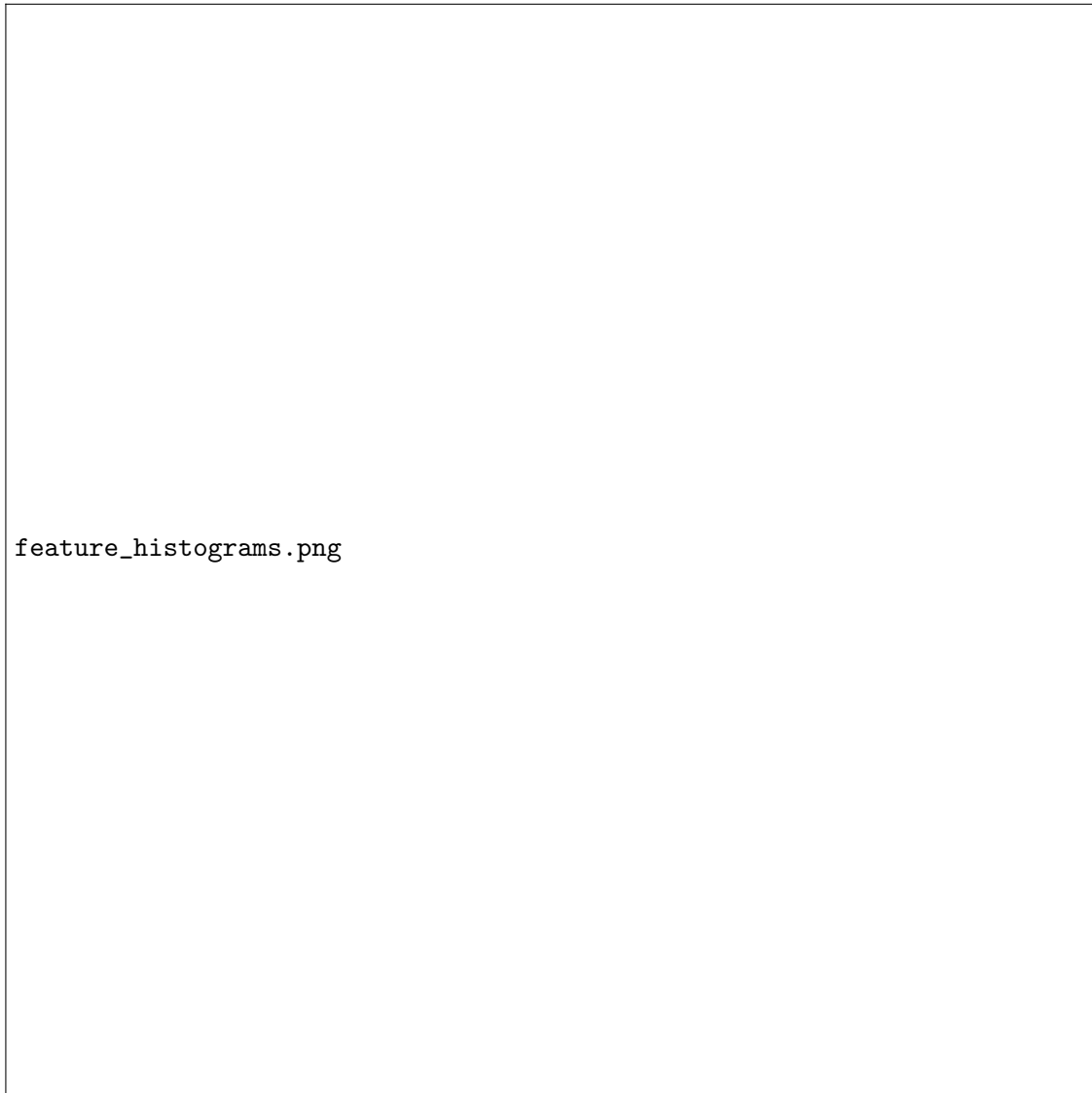


Figure 1: Feature Histograms

Interpretation:

The histograms show the distributions of Variance, Skewness, Kurtosis, and Entropy. The shapes of the distributions help identify skewness, concentration of observations, and possible extreme values.

8.2 Correlation Heatmap

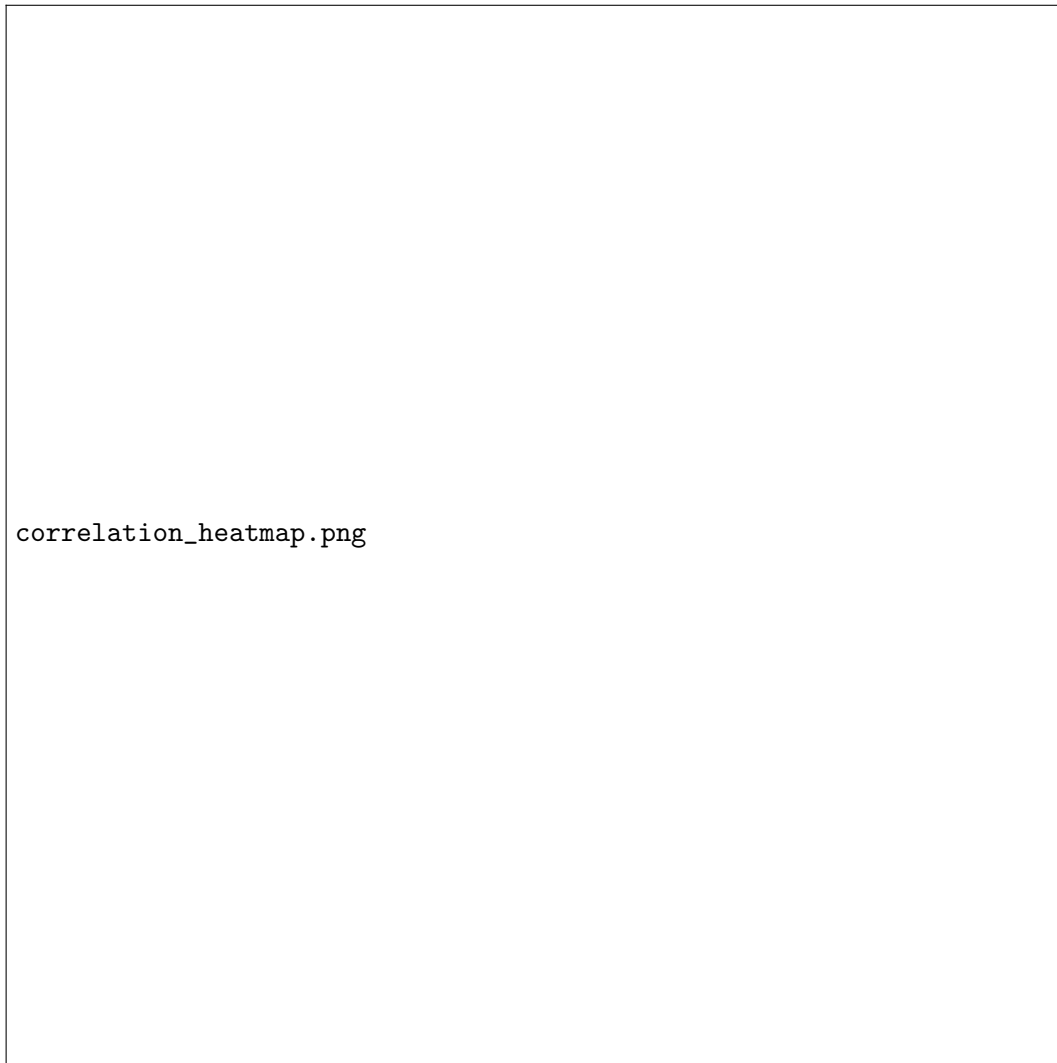


Figure 2: Correlation Heatmap

Interpretation:

The heatmap displays pairwise linear correlations among the variables. Large positive or negative coefficients indicate stronger linear relationships, while values close to zero indicate weak linear relationships.

8.3 Scatter Plot



Figure 3: Class Distribution Scatter Plot

Interpretation:

Each point represents one banknote sample using two selected features. The class colours help assess whether the two classes appear approximately linearly separable in the selected two-dimensional view.

8.4 Training Error vs Epoch

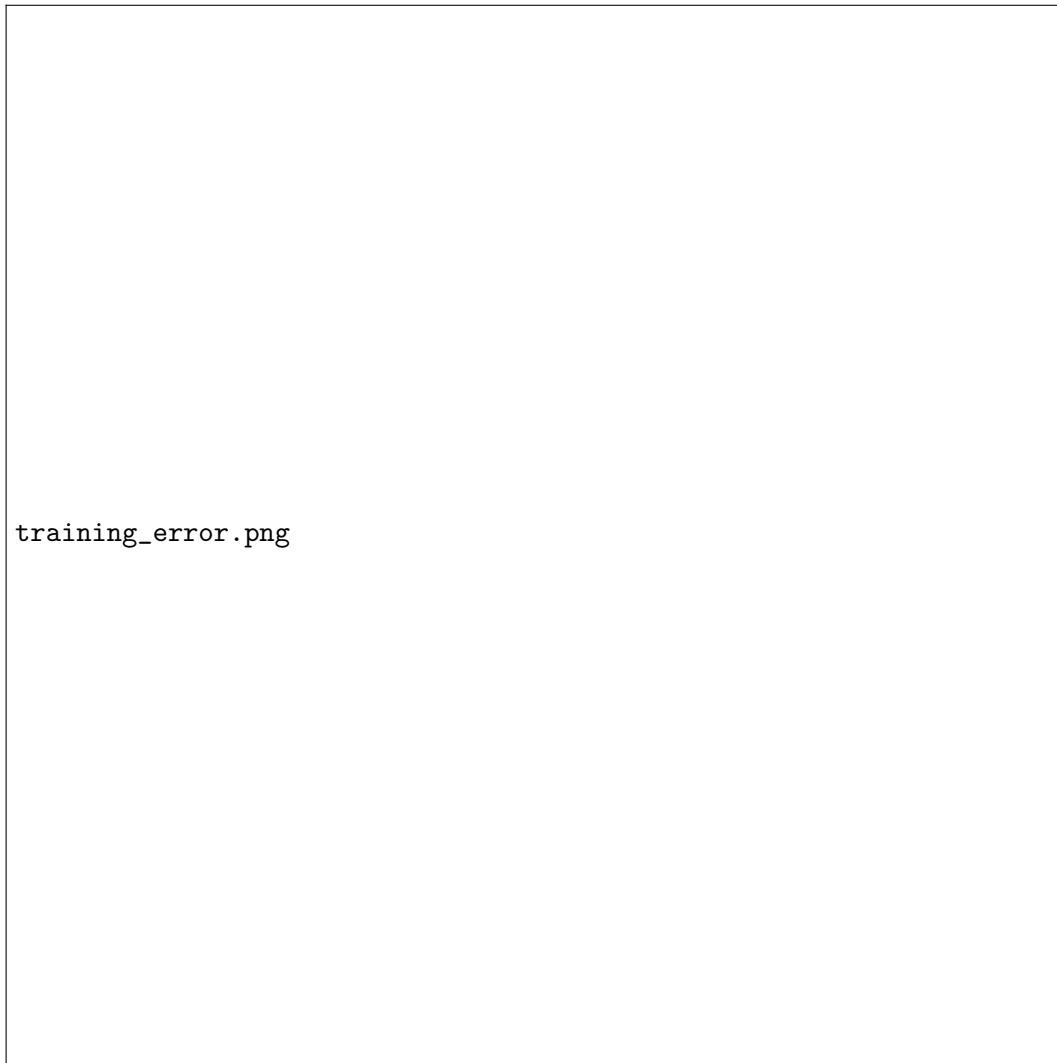


Figure 4: Training Error vs Epoch

Interpretation:

This plot shows the number of misclassified training samples after each epoch. The trend is used to study whether the perceptron approaches stable learning behaviour.

8.5 Weight Evolution



Figure 5: Weight Evolution

Interpretation:

The plot shows how the four feature weights change across training epochs. Stabilization of the curves indicates that the learned parameters are becoming less variable.

8.6 Bias Evolution



Figure 6: Bias Evolution

Interpretation:

The plot shows how the bias parameter changes during training. The bias shifts the linear decision boundary and is updated whenever a sample is misclassified.

8.7 Confusion Matrix

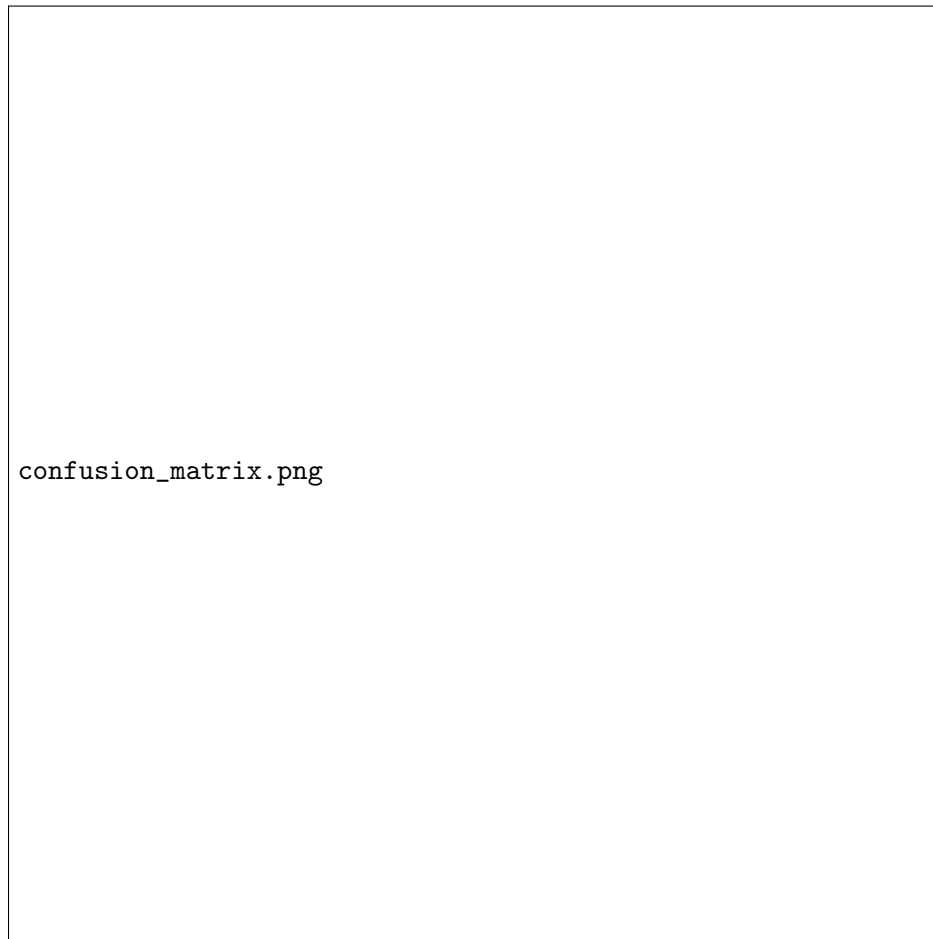


Figure 7: Confusion Matrix

Interpretation:

The confusion matrix summarizes correct and incorrect predictions. Its cells represent True Positives, True Negatives, False Positives, and False Negatives.

8.8 Learning Rate Comparison

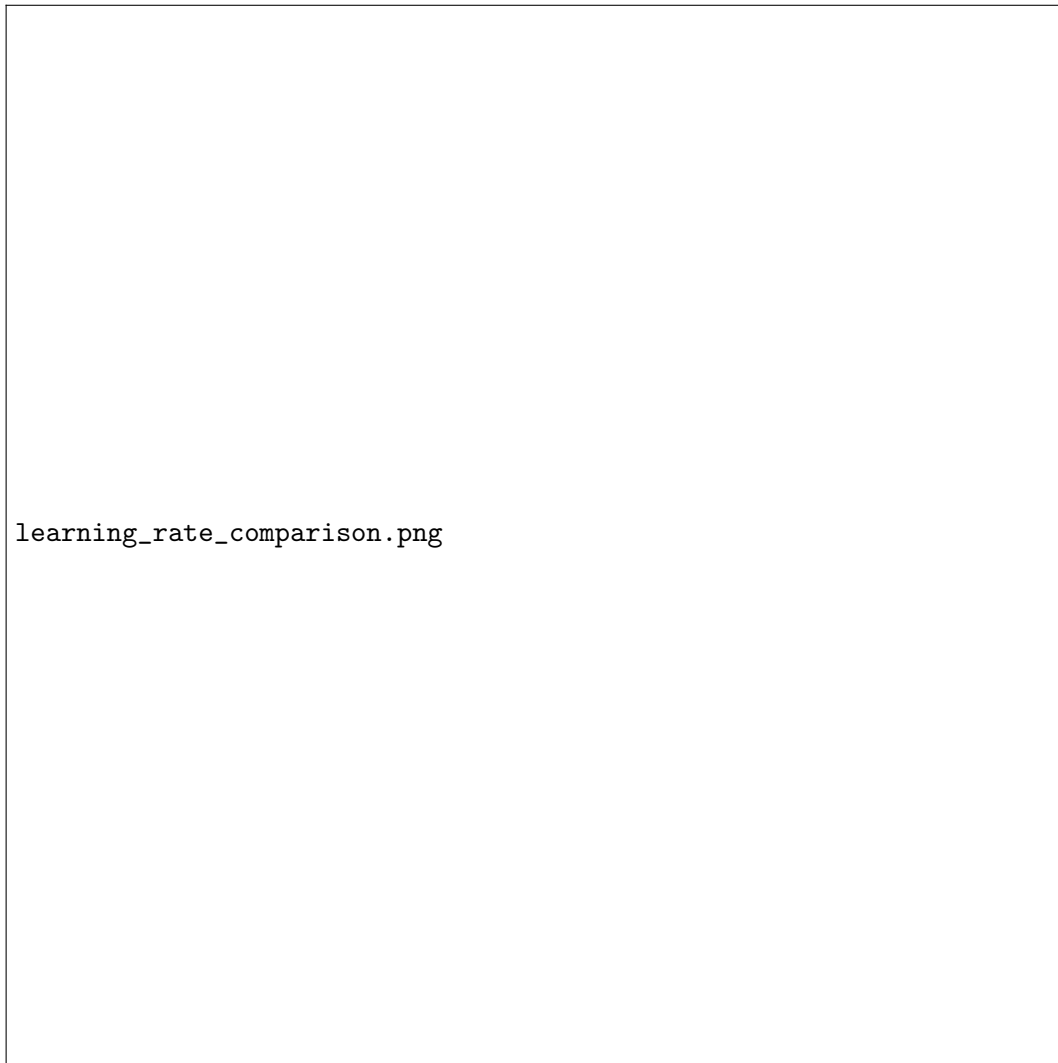


Figure 8: Learning Rate Comparison

Interpretation:

The curves compare training behaviour for learning rates 0.001, 0.01, and 0.1. Their error patterns are used to study how update magnitude affects convergence behaviour.

8.9 Step vs Sigmoid Comparison



Figure 9: Step and Sigmoid Activation Functions

Interpretation:

The Step function changes abruptly between 0 and 1, whereas Sigmoid changes smoothly. The absence of a useful gradient makes the Step function unsuitable for ordinary gradient-based backpropagation.

8.10 XOR Experiment

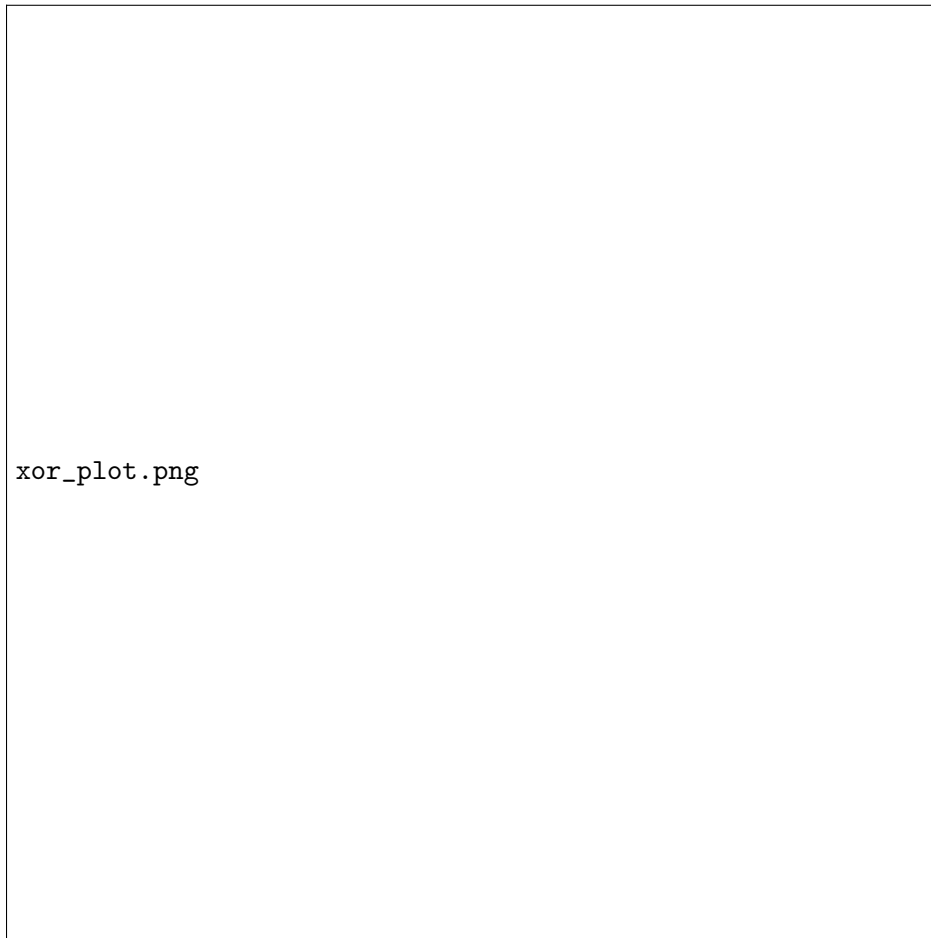


Figure 10: XOR Class Distribution

Interpretation:

The XOR classes are arranged diagonally in the input space. No single straight line can separate the two classes, demonstrating why a Single Layer Perceptron cannot perfectly solve XOR.

8.11 Normalization Study



Figure 11: Effect of Feature Normalization on Convergence

Interpretation:

The plot compares training errors with and without feature normalization. The actual curves should be used to determine whether standardization improved stability or convergence speed in this run.

9 Performance Tables

9.1 Training Summary

Parameter	Value
Dataset Size	1372
Train/Test Split	80% / 20%
Learning Rate	0.01
Epochs	50
Final Weights	<i>Insert actual values</i>
Final Bias	<i>Insert actual value</i>
Accuracy	<i>Insert actual value</i>
Precision	<i>Insert actual value</i>
Recall	<i>Insert actual value</i>
F1-score	<i>Insert actual value</i>

Table 2: Training Summary

9.2 Epoch-wise Learning

Epoch	Errors	W1	W2	W3	W4	Bias
1	–	–	–	–	–	–
2	–	–	–	–	–	–
3	–	–	–	–	–	–

9.3 Learning Rate Performance Comparison

Learning Rate	Accuracy	Precision	Recall	F1-score
0.001	–	–	–	–
0.01	–	–	–	–
0.1	–	–	–	–

Table 4: Learning Rate Performance Comparison

9.4 Normalization Performance Comparison

Model	Accuracy	Precision	Recall	F1-score
Without Normalization	–	–	–	–
With Normalization	–	–	–	–

Table 5: Normalization Performance Comparison

9.5 Custom vs Scikit-learn Comparison

Model	Accuracy	Precision	Recall	F1-score
From Scratch	–	–	–	–
Scikit-learn	–	–	–	–

Table 6: Custom and Scikit-learn Perceptron Comparison

10 Discussion

The Single Layer Perceptron learns by modifying its weights and bias whenever a training sample is misclassified. The training-error, weight-evolution, and bias-evolution plots provide information about the learning behaviour and parameter changes across epochs.

The learning-rate study compares update magnitudes of 0.001, 0.01, and 0.1. The final interpretation should be based on the observed convergence curves and test metrics rather than assuming that a larger or smaller learning rate is always superior.

The Step and Sigmoid comparison demonstrates an important distinction between classical perceptron learning and modern gradient-based neural network training. The Step function is suitable for hard binary decisions but does not provide useful gradients for backpropagation.

The XOR experiment demonstrates the fundamental limitation of a Single Layer Perceptron. XOR is not linearly separable, and therefore a single linear decision boundary cannot perfectly classify all XOR samples. A multilayer network with hidden neurons can overcome this limitation.

The normalization study investigates how input scaling affects perceptron updates. Since the update rule directly contains the input vector, features with different numerical magnitudes can produce different update scales. The actual error curves and metrics determine whether normalization improved convergence in this experiment.

The comparison with Scikit-learn provides a reference against a standard library implementation. Differences may occur because of implementation details such as stopping criteria, sample ordering, regularization defaults, and other training behaviour.

11 Conclusion

A Single Layer Perceptron was implemented from scratch and applied to the Banknote Authentication binary classification problem. The experiment covered dataset exploration, exploratory data analysis, feature normalization, train-test splitting, perceptron training, and performance evaluation.

The learning process was studied using training errors, weight evolution, bias evolution, and confusion-matrix analysis. Additional studies compared different learning rates, Step and Sigmoid activation functions, normalized and non-normalized training, and the custom model with Scikit-learn.

The XOR experiment demonstrated that a Single Layer Perceptron is limited to linearly separable problems. Overall, the experiment provides a foundation for understanding artificial neurons, linear classifiers, learning rules, activation functions, and the motivation for multilayer neural networks.

12 References

1. F. Rosenblatt, “The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain,” *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson.
5. UCI Machine Learning Repository, Banknote Authentication Dataset.
6. Scikit-learn Documentation, Perceptron and StandardScaler.